

INSTITUT DE TECHNOLOGIES APPLIQUÉES DES CAYES (ITAC)

Département Informatique

DÉVELOPPEMENT MOBILE AVEC FLUTTER

CAHIER DES CHARGES

Projet Final

SocialSnap

Application Mobile de Partage de Photos

Réalisé par

Jocelyn Youvens Dions
Faustin Mardoche
Eserve Wondy

Encadré par

Ing. Phawens LOUIS-JEAN

Année Académique

2025 – 2026

CAHIER DES CHARGES

SocialSnap

Réseau Social de Partage de Photos — Application Mobile Flutter & Firebase

Cours : DEVELOPPEMENT MOBILE AVEC FLUTTER

Établissements : ITAC

Projet Final N° 01

Rôle	Membre de l'équipe	Responsabilité
Développement de l'application	Jocelyn Youvens Dions	Développement Flutter / Firebase (Front-End & Back-End)
Conception des maquettes	Faustin Mardoche	Design UI/UX, maquettes et identité visuelle
Rédaction du Cahier des Charges	Eserve Windy	Rédaction, structuration et validation du CDC

Champ	Information
Nom du projet	SocialSnap
Client / Commanditaire	DEVELOPPEMENT MOBILE AVEC FLUTTER
Équipe projet	Jocelyn Youvens Dions, Faustin Mardoche, Eserve Windy
Version du document	v1.0
Date de création	30/06/26
Dernière mise à jour	10/07/26
Statut du document	complet
Type de CDC	CDC Technique (fonctionnel + technique)

Gestion des Versions

Historique des modifications apportées au présent document.

Version	Auteur	Date	Description des modifications
v0.1	Eserve Wondy	01/07/26	Création initiale du document — structure et trame générale
v0.5	Eserve Wondy	03/07/26	Intégration des besoins fonctionnels et non fonctionnels détaillés
v1.0	Eserve Wondy	06/07/26	Version soumise à validation — relecture par l'équipe

1. Introduction et Contexte Général

1.1. Contexte de l'entreprise / du projet

SocialSnap est un projet académique réalisé dans le cadre du cours de DEVELOPPEMENT MOBILE AVEC FLUTTER Haïti connaît une pénétration croissante des smartphones et des réseaux sociaux. Les plateformes existantes (Instagram, Facebook) sont majoritairement en anglais et ne répondent pas pleinement aux spécificités culturelles, linguistiques et sociales du pays. Il existe donc une opportunité de concevoir une application de réseau social photo adaptée au contexte local, supportant le français et le créole haïtien.

1.2. Problématique

Comment concevoir et développer une application mobile de partage de photos offrant une expérience sociale complète (publication, interaction, découverte, messagerie), synchronisée en temps réel et sécurisée, en utilisant Flutter et l'écosystème Firebase ?

1.3. Objectifs du projet (SMART)

- Spécifique : développer une application mobile Android/iOS de partage de photos avec fonctionnalités sociales complètes (profils, abonnements, likes, commentaires, messagerie temps réel, notifications).
- Mesurable : feed chargé en moins de 2 secondes sur 3G, couverture des 4 modules principaux (Authentification, Publication, Social, Messagerie).
- Atteignable : stack technique maîtrisée par l'équipe (Flutter, Firebase, Provider) sur une période de 4 semaines.
- Réaliste : périmètre fonctionnel calibré pour une équipe de 3 personnes et une soutenance académique.
- Temporel : livraison finale (APK + soutenance) à l'issue de la semaine 4 du planning prévisionnel.

1.4. Périmètre inclus / exclu

Inclus dans le périmètre (In Scope)

- Authentification complète (inscription, connexion, déconnexion, vérification email, réinitialisation mot de passe).
- Gestion de profils utilisateurs (photo, displayName, username, biographie).
- Système d'abonnement Follow / Unfollow.
- Publication de photos avec légende, hébergement Firebase Storage ou ImgBB.
- Fil d'actualité (Feed) personnalisé selon les abonnements.
- Page de recherche / exploration Bento avec 3 filtres (Tout, People, Posts).
- Likes en temps réel et commentaires avec fil de réponses.
- Notifications in-app (like, commentaire, nouvel abonné).
- Messagerie privée en temps réel avec gestion des messages non lus (badges).
- Mode sombre / mode clair (Dark Mode / Light Mode) via Provider.

Hors périmètre (Out of Scope)

- Stories éphémères avec expiration automatique après 24h (prévu en version future).
- Système de filtres photo avancés / retouche d'image intégrée.
- Appels audio / vidéo dans la messagerie.
- Monétisation, publicités ou boutique intégrée.

- Version web de l'application (le périmètre se limite à Android/iOS).

1.5. Glossaire

Terme	Définition
CDC	Cahier des Charges
Feed	Fil d'actualité affichant les publications des comptes suivis
Follow / Unfollow	Action de s'abonner ou se désabonner d'un autre utilisateur
StreamBuilder	Widget Flutter qui reconstruit l'UI en écoutant un flux de données en temps réel
Provider	Solution de gestion d'état utilisée pour séparer la logique métier de l'interface
Firestore	Base de données NoSQL temps réel de Firebase
WriteBatch	Mécanisme Firestore permettant d'effectuer plusieurs écritures de façon atomique
Bento UI	Style de mise en page en grille modulaire inspiré des boîtes bento japonaises
MVVM	Model-View-ViewModel, patron d'architecture séparant données, logique et présentation

2. Description de l'Existant (État des lieux)

Avant la conception de SocialSnap, l'équipe a analysé le contexte existant afin de justifier les choix fonctionnels et techniques du projet.

- Processus actuel : les utilisateurs haïtiens partagent leurs photos principalement via des plateformes internationales (Instagram, Facebook, WhatsApp Statuts), sans solution locale dédiée.
- Systèmes existants : aucun système interne préexistant ; le projet part d'une base entièrement nouvelle (greenfield).
- Points de douleur identifiés : interfaces en anglais, absence de personnalisation culturelle, manque d'options en créole haïtien.
- Volume de données à migrer : aucun (nouveau projet, pas de migration de données).
- Contraintes héritées : aucune contrainte de système legacy ; liberté totale sur le choix de la stack technique (Flutter + Firebase imposés par le cadre pédagogique du cours).

3. Besoins Fonctionnels

Les besoins fonctionnels sont organisés par module et priorisés selon la méthode MoSCoW (Must Have, Should Have, Could Have, Won't Have).

3.1. Module Authentification & Profils

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-001	L'utilisateur doit pouvoir s'inscrire via email, mot de passe et pseudo unique	Must Have	Utilisateur final	Compte créé en < 2 min, pseudo unique vérifié
BF-002	Le système doit envoyer un email de vérification à l'inscription	Must Have	Système	Email reçu en < 5 min
BF-003	L'utilisateur doit pouvoir se connecter et se déconnecter de façon sécurisée	Must Have	Utilisateur final	Session active persistée via authStateChanges
BF-004	L'utilisateur doit pouvoir réinitialiser son mot de passe par email	Must Have	Utilisateur final	Email de réinitialisation envoyé et fonctionnel
BF-005	L'utilisateur doit pouvoir modifier son profil (photo, displayName, username, bio)	Must Have	Utilisateur final	Modifications reflétées immédiatement sur le profil
BF-006	Le système doit afficher le profil d'un utilisateur (photo, nom, bio, publications)	Must Have	Utilisateur final	Page profil accessible en un clic depuis n'importe quel post
BF-007	L'utilisateur doit pouvoir suivre / ne plus suivre (Follow/Unfollow) un autre utilisateur	Must Have	Utilisateur final	Compteurs abonnés/abonnements mis à jour en temps réel

3.2. Module Publication (Posts)

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-008	L'utilisateur doit pouvoir capturer une photo via l'appareil photo natif	Must Have	Utilisateur final	Photo capturée et prévisualisée avant publication
BF-009	L'utilisateur doit pouvoir sélectionner une photo depuis la galerie	Must Have	Utilisateur final	Sélection via image_picker fonctionnelle
BF-010	L'utilisateur doit pouvoir ajouter une légende (caption) à sa publication	Must Have	Utilisateur final	Légende enregistrée et affichée avec le post
BF-011	Le système doit héberger l'image sur Firebase Storage ou ImgBB	Must Have	Système	URL publique générée et stockée dans Firestore
BF-012	Le système doit horodater chaque publication et la lier à son auteur	Must Have	Système	Champ createdAt et uid présents sur chaque post

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-013	L'auteur doit pouvoir modifier ou supprimer sa propre publication	Should Have	Utilisateur final	Suppression confirmée par dialogue, retrait immédiat de la grille
BF-014	Le système doit afficher la publication dans le fil des abonnés de l'auteur	Must Have	Système	Post visible dans le feed de chaque abonné < 5s après publication

3.3. Module Social (Feed, Likes, Commentaires, Notifications)

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-015	Le système doit afficher un fil d'actualité personnalisé selon les abonnements	Must Have	Système	Feed via StreamBuilder, tri par date décroissante
BF-016	L'utilisateur doit pouvoir liker / unliker une publication en temps réel	Must Have	Utilisateur final	Compteur de likes mis à jour instantanément, stocké en sous-collection
BF-017	L'utilisateur doit pouvoir commenter une publication	Must Have	Utilisateur final	Commentaire visible immédiatement sous le post
BF-018	L'utilisateur doit pouvoir répondre à un commentaire (fil de réponses)	Should Have	Utilisateur final	Réponse affichée en retrait sous le commentaire parent
BF-019	Le système doit générer une notification in-app pour chaque like, commentaire, abonné	Must Have	Système	Notification créée et visible < 3s après l'action
BF-020	Le système doit proposer une page Découverte / Recherche avec filtres Tout/People/Posts	Should Have	Utilisateur final	3 filtres fonctionnels, navigation fluide entre vues
BF-021	L'utilisateur doit pouvoir rechercher un autre utilisateur par pseudo ou nom	Must Have	Utilisateur final	Résultats filtrés en temps réel pendant la saisie

3.4. Module Messagerie en Temps Réel

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-022	Deux utilisateurs doivent pouvoir échanger des messages privés instantanés	Must Have	Utilisateur final	Message livré et affiché en < 2s
BF-023	Le système doit utiliser un WriteBatch pour écrire le message et mettre à jour les métadonnées du chat de façon atomique	Must Have	Système	Aucune incohérence entre message et dernier message affiché
BF-024	Le système doit afficher un badge rouge dynamique en cas de message non lu	Must Have	Système	Badge visible sur l'icône messagerie quand lastSenderId != currentUid et isRead =

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
				false
BF-025	Le statut isRead doit passer à true dès l'ouverture de la conversation	Must Have	Système	Badge disparaît instantanément à l'ouverture du chat
BF-026	Le système devrait afficher un indicateur de saisie en cours (« ... is typing »)	Could Have	Système	Indicateur visible via typing_model en temps réel

3.5. Module Paramètres & Apparence

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-027	L'utilisateur doit pouvoir activer/désactiver le mode sombre (Dark Mode)	Should Have	Utilisateur final	Bascule immédiate de toute l'interface via theme_provider
BF-028	L'utilisateur doit pouvoir changer son mot de passe depuis les paramètres	Should Have	Utilisateur final	Confirmation de changement, email de sécurité si applicable

4. Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les qualités attendues du système, indépendamment des fonctionnalités.

Catégorie	Exigences
Performance	Le feed doit charger en moins de 2 secondes sur une connexion 3G. Pagination Firestore (startAfter, 12 posts par page).
Disponibilité	Disponibilité visée : 99% (infrastructure Firebase managée), avec gestion explicite des états hors-ligne.
Sécurité	Règles de sécurité Firestore garantissant qu'un utilisateur ne peut modifier que ses propres données (posts, profil, messages).
Ergonomie / UX	Interface fluide avec gestion explicite des états loading / empty / error sur chaque écran.
Accessibilité	Contrastes suffisants en mode clair et sombre ; tailles de police lisibles ; cibles tactiles ≥ 44 px.
Maintenabilité	Architecture MVVM + Provider, séparation models/services/providers/screens/widgets, code documenté.
Portabilité	Application compatible Android et iOS via Flutter 3.x, design responsive sur différentes tailles d'écran.
Scalabilité	Structure Firestore en collections/sous-collections pensée pour supporter la croissance du nombre d'utilisateurs sans refonte majeure.
Optimisation médias	Compression des images avant upload (max 800x800px, qualité 80%) via flutter_image_compress.

5. Contraintes du Projet

- Contraintes budgétaires : projet académique sans budget alloué ; utilisation exclusive des offres gratuites (Firebase Spark Plan, ImgBB gratuit).
- Contraintes temporelles : livraison finale (APK + soutenance) en fin de semaine 4 du planning prévisionnel.
- Contraintes techniques : stack imposée par le cadre pédagogique — Flutter 3.x (Dart, null safety) et écosystème Firebase (Auth, Firestore, Storage).
- Contraintes réglementaires : conformité aux bonnes pratiques de protection des données personnelles (mots de passe non stockés en clair, gestion via Firebase Authentication).
- Contraintes organisationnelles : équipe de 3 membres avec rôles définis (développement, design, rédaction CDC) ; disponibilité limitée aux heures de cours et de travail personnel.
- Contraintes de ressources humaines : compétences Flutter/Firebase en cours d'acquisition dans le cadre du cours ; accompagnement pédagogique du professeur.

6. Architecture et Spécifications Techniques

6.1. Architecture applicative

L'application suit le patron d'architecture MVVM (Model-View-ViewModel) adapté à Flutter via le package Provider. Cette architecture sépare clairement les données (models), l'accès aux données (services), la gestion d'état (providers) et la présentation (screens/widgets).

6.2. Stack technologique

Composant	Technologie / Package	Rôle dans le projet
Langage	Dart 3.x (null safety)	Toute la logique métier et l'interface utilisateur
Framework UI	Flutter 3.x — Material 3	Construction de tous les écrans de l'application
Authentification	firebase_auth ^4.x	Gestion des sessions et de l'identité utilisateur
Base de données	cloud_firestore ^4.x	Stockage NoSQL temps réel (profils, posts, chats, notifications)
Stockage fichiers	firebase_storage ^11.x	Hébergement des photos de profil et de publication
Gestion d'état	provider ^6.x + ChangeNotifier	État global de l'application (Auth, Feed, Chat, Thème, Notifications)
Caméra	camera ^0.10.x	Capture photo native
Galerie	image_picker ^1.x	Sélection d'images depuis la galerie
Compression images	flutter_image_compress ^2.x	Compression avant upload (800x800px, qualité 80%)
Temps réel	Stream + StreamBuilder (Flutter)	Mise à jour de l'UI sans rechargement manuel
Navigation	GoRouter ^12.x	Routing déclaratif et gestion des deep links
Hébergement alternatif images	ImgBB API (http package)	Solution d'hébergement d'images alternative à Firebase Storage

6.3. Modèle de données Firestore

La base de données NoSQL est organisée autour des collections principales suivantes :

Collection	Champ	Type	Description
users/{uid}	uid	String	Identifiant Firebase Auth
	username	String	Nom d'utilisateur unique
	displayName	String	Nom affiché publiquement
	bio	String	Description du profil

Collection	Champ	Type	Description
	profileImageUrl	String	URL avatar (Storage / ImgBB)
	followersCount	Number	Compteur d'abonnés
	followingCount	Number	Compteur d'abonnements
	createdAt	Timestamp	Date de création du compte
posts/{postId}	uid	String	Référence vers users/{uid}
	imageUrl	String	URL de la photo
	caption	String	Légende de la publication
	likesCount	Number	Compteur de likes
	commentsCount	Number	Compteur de commentaires
	createdAt	Timestamp	Pour tri chronologique
posts/{id}/likes/{uid}	uid	String	UID de l'utilisateur ayant liké
posts/{id}/comments/{cid}	uid, text, replyToCommentId	String	Commentaire et fil de réponses
chats/{chatId}	participants	Array	UIDs des deux utilisateurs
	lastMessage	String	Dernier message échangé
	lastSenderId	String	UID du dernier émetteur
	isRead	Boolean	Statut de lecture du dernier message
chats/{id}/messages/{mid}	senderId, text, createdAt	Mixte	Sous-collection des messages
notifications/{nid}	recipientId, type, actorId	String	Type : like comment follow
	postId	String?	Optionnel (si lié à un post)
	isRead	Boolean	Lu ou non lu

6.4. Architecture du code (dossiers réels du projet)

L'arborescence du projet Flutter est structurée comme suit, en cohérence avec le patron MVVM + Provider :

Dossier	Contenu
---------	---------

Dossier	Contenu
models/	7 modèles : user_model, post_model, comment_model, chat_model, message_model, notification_model, typing_model
services/	auth_service, firestore_service, storage_service — couche d'accès aux données
providers/	auth_provider, feed_provider, chat_provider, notification_provider, theme_provider
screens/auth	Écrans login et inscription
screens/home	Fil d'actualité et CustomGlassBottomBar
screens/profile	Profil utilisateur et paramètres (Glassmorphism)
screens/create	Création de publication (caméra / galerie)
screens/post	Détail d'une publication et commentaires
screens/search	Page d'exploration Bento (filtres Tout / People / Posts)
screens/chat	Liste des discussions et chambre de conversation
screens/notifications	Liste des activités (likes, commentaires, abonnés)
widgets/	Composants réutilisables : PostCard, StoryCircle, CustomGlassBottomBar, GlowButton, etc.
main.dart	Point d'entrée — MultiProvider racine + configuration GoRouter

7. Exigences de Sécurité et Conformité

- Authentification gérée exclusivement via Firebase Authentication (email/mot de passe), aucun mot de passe stocké en clair côté application.
- Règles de sécurité Firestore : un utilisateur ne peut lire/écrire que les documents lui appartenant (posts, profil, messages dont il est participant).
- Vérification d'email obligatoire avant accès complet aux fonctionnalités sociales.
- Journalisation implicite via les champs createdAt sur chaque document pour assurer la traçabilité des actions.
- Aucune donnée personnelle sensible (autre que email, pseudo, bio) n'est collectée.
- Tests de sécurité de base : vérification manuelle que les règles Firestore empêchent la modification de données appartenant à un autre utilisateur.

8. Livrables Attendus

Livrable	Description
Code source	Dépôt GitHub complet, historique de commits, branches feature/organisées
Application (APK)	Fichier APK final installable, compilé en mode release
Documentation technique	README avec architecture, instructions d'installation et captures d'écran
Cahier des Charges	Le présent document, validé par l'équipe et le professeur
Maquettes	Maquettes visuelles (Figma ou équivalent) servant de référence design
Modèle de données	Schéma des collections Firestore documenté (présente section 6.3)
Soutenance	Démonstration en direct de l'application devant le jury académique

9. Critères de Recette et de Validation

9.1. Procédure de recette fonctionnelle

Chaque membre de l'équipe teste les modules dont il n'est pas l'auteur principal (test croisé), selon les scénarios définis dans le tableau des besoins fonctionnels (section 3). Le professeur valide la recette finale lors de la soutenance.

9.2. Critères de rejet

- Crash de l'application lors d'un scénario nominal (publication, like, commentaire, message).
- Non-respect d'un besoin Must Have du tableau des besoins fonctionnels.
- Absence de gestion des états loading / error / empty sur un écran principal.

9.3. Période de garantie / support

Dans le cadre académique, une période de correction est prévue entre la remise du livrable et la soutenance, durant laquelle l'équipe peut corriger les anomalies signalées par le professeur.

10. Planning Prévisionnel

Phase	Tâches à réaliser	Livrable
S1 — Initialisation	Création du dépôt GitHub, configuration Firebase (Auth, Firestore, Storage), mise en place de GoRouter et MultiProvider, écran login/register fonctionnel	Auth Firebase OK + dépôt GitHub initialisé
S2 — Fonctionnalités principales	Modèle de données Firestore, publication de photo (caméra + galerie + upload Storage), FirestoreService, FeedProvider	Publication visible en base de données
S3 — Couche sociale	Feed personnalisé (Stream), Follow/Unfollow, likes temps réel (sous-collection), commentaires, page Profil, messagerie en temps réel	Feed + interactions sociales fonctionnels
S4 — Finalisation & livraison	Notifications in-app, page Découverte (filtres Bento), pagination, gestion des états, compression d'images, tests, APK final	APK final + soutenance

11. Conditions Académiques et Organisationnelles

Le présent document étant rédigé dans un cadre pédagogique et non commercial, les clauses contractuelles classiques sont adaptées au contexte académique :

- Évaluation : le projet est noté selon la grille d'évaluation spécifique du cours (architecture, intégration Firebase, fonctionnalités sociales, caméra/médias, UX, GitHub, soutenance).
- Propriété intellectuelle : le code source reste la propriété de l'équipe étudiante, à des fins de portfolio académique.
- Confidentialité : aucune donnée réelle d'utilisateur n'est utilisée ; seules des données de test sont manipulées durant le développement.
- Responsabilités : chaque membre de l'équipe est responsable de son rôle assigné (développement, design, documentation) et doit pouvoir défendre sa contribution à la soutenance.
- Gestion des évolutions : toute modification du périmètre fonctionnel après validation du présent CDC doit être consignée dans le tableau de gestion des versions.

12. Annexe — Grille d'Évaluation Spécifique

Critère	Indicateurs de réussite	Points
Architecture du code (MVVM + Provider)	Séparation claire models/services/providers/screens, ChangeNotifier correctement implémenté, 0 logique métier dans les widgets	20 pts
Intégration Firebase (Auth + Firestore + Storage)	Auth persistante, CRUD Firestore sécurisé par règles, upload Storage avec progression, StreamBuilder pour le temps réel	25 pts
Fonctionnalités sociales	Feed personnalisé fonctionnel, Follow/Unfollow, likes temps réel, commentaires, notifications in-app	20 pts
Caméra & médias	Capture caméra native, galerie, compression avant upload, aperçu avant publication	10 pts
UX & design	Navigation fluide, états loading/error/empty gérés, design cohérent, responsive	10 pts
GitHub & méthodologie	Minimum 3 branches feature/, commits réguliers (min. 15), README avec captures d'écran	10 pts
Soutenance & défense	Chaque membre explique son code, questions techniques répondues, démo en direct sans crash	5 pts

TOTAL : 100 points

13. Rappel — Bonnes Pratiques de Rédaction

Conformément au guide méthodologique du cours, l'équipe veille à :

- Quantifier toutes les exigences plutôt que d'utiliser des formulations vagues (« rapide », « simple »).
- Identifier chaque besoin par un ID unique et traçable (BF-001, BF-002, ...).
- Définir les critères de recette avant le début du développement, et non après.
- Impliquer les trois membres de l'équipe dans la validation finale du document.
- Archiver chaque version du CDC dans le tableau de gestion des versions (section « Gestion des Versions »).